



# **CSE 4513**

## **Lec – 8**

# **Software Design**

# DEPENDENCY INVERSION PRINCIPLE



✓ High-level modules should not depend on low-level modules. Instead, both should depend on **abstractions (like interfaces or abstract classes)**. Abstractions should not depend on details.

Details should depend on abstractions.



most flexible systems are those in which source code dependencies refer only to abstractions.

# WHAT IS DEPENDENCY



A dependency is something - another class, or a value - that your class depends on, or requires. For example:

- ✓ If a class requires some setting, it might call

*ConfigurationManager.AppSettings["someSetting"]*.

*ConfigurationManager* is a dependency. The class has a dependency on *ConfigurationManager*.

- ✓ If you write two classes - let's say *Log* and *SqlLogWriter* - and *Log* creates or requires an instance of *SqlLogWriter*, then *Log* depends on *SqlLogWriter*.  
*SqlLogWriter* is a dependency.

# DEPENDENCIES....



- ✓ Framework
- ✓ Third Party Libraries
- ✓ Database
- ✓ File System
- ✓ Email
- ✓ Web Services
- ✓ System Resources (Clock)
- ✓ Configuration
- ✓ The new Keyword
- ✓ Static methods
- ✓ Thread.Sleep
- ✓ Random

# HIGH LEVEL MODULES AND LOW LEVEL MODULES

---



The principle says that “both should depend on abstractions.” means that when it comes to applying the principle, the difference between high level and low level doesn’t matter.

# WHAT ARE ABSTRACTIONS?



- ✓ Typically, **interfaces or abstract classes** serve as abstractions, as they define methods and properties without implementing the actual logic.
- ✓ **Purpose:**
  - ✓ Abstractions define "what" something does without specifying "how" it does it.
- ✓ **Example:**
  - ✓ An interface `PaymentProcessor` could represent an abstraction for any kind of payment processor. It might define a method `processPayment`, which different payment methods can implement.

```
// Abstraction: PaymentProcessor interface defines a common behavior  
public interface PaymentProcessor {  
    void processPayment(double amount);  
}
```

# DIP: LET'S LOOK AT A VERY SIMPLE EXAMPLE



- ✓ Suppose you are building a simple system for sending messages. Initially, you have two classes: **SMS** and **Email** for sending SMS and email messages, respectively:

```
class SMS {  
    public void sendSMS(String message) {  
        // Code to send an SMS  
    }  
}  
  
class Email {  
    public void sendEmail(String message) {  
        // Code to send an email  
    }  
}
```

# DIP: LET'S LOOK AT A VERY SIMPLE EXAMPLE



- ✓ You also have a high-level class called **MessageSender** that uses these classes to send

messages:

```
class MessageSender {  
    private SMS sms;  
    private Email email;  
  
    public MessageSender() {  
        sms = new SMS();  
        email = new Email();  
    }  
  
    public void sendSMS(String message) {  
        sms.sendSMS(message);  
    }  
  
    public void sendEmail(String message) {  
        email.sendEmail(message);  
    }  
}
```

- ✓ In this design, the **MessageSender** class directly depends on the low-level classes **SMS** and **Email**.
- ✓ This violates the DIP because high-level modules should not depend on low-level modules.

# DIP: LET'S LOOK AT A VERY SIMPLE EXAMPLE



- ✓ To apply DIP, we can introduce an abstraction (interface) that both **SMS** and **Email** will depend on.
- ✓ That will allow the **MessageSender** class to depend on the abstraction instead of the concrete

implementations:

```
interface MessageService {
    void sendMessage(String message);
}

class SMS implements MessageService {
    public void sendMessage(String message)
    {
        // Code to send an SMS
    }
}

class Email implements MessageService {
    public void sendMessage(String message)
    {
        // Code to send an email
    }
}
```

# DIP: LET'S LOOK AT A VERY SIMPLE EXAMPLE



- ✓ Now, the **SMS** and **Email** classes implement the **MessageService** interface. We've inverted the dependency, making high-level **MessageSender** depend on the abstraction **MessageService**:

```
class MessageSender {  
    private MessageService messageService;  
  
    public MessageSender(MessageService messageService) {  
        this.messageService = messageService;  
    }  
  
    public void sendMessage(String message) {  
        messageService.sendMessage(message);  
    }  
}
```

With this change, the **MessageSender** class is no longer tightly coupled to the concrete implementations of sending messages. It can work with any class that implements the **MessageService** interface.

# LET'S LOOK AT THE CODE



```
// Low-level module: specific payment processor
class CreditCardProcessor {
    public void processPayment(double amount) {
        System.out.println("Processing credit card payment of $" + amount);
    }
}

// High-level module: payment service
class PaymentService {
    private CreditCardProcessor processor;

    public PaymentService(CreditCardProcessor processor) {
        this.processor = processor;
    }

    public void makePayment(double amount) {
        processor.processPayment(amount);
    }
}
```

*PaymentService is directly dependent on  
CreditCardProcessor, violating DIP.*

# APPLYING DIP



To apply DIP, we can introduce an interface `PaymentProcessor` as an abstraction that `PaymentService` will depend on. Then we can create multiple implementations (like `CreditCardProcessor` and `PaypalProcessor`) that implement the `PaymentProcessor` interface.

```
// Abstraction: payment processor interface
interface PaymentProcessor {
    void processPayment(double amount);
}

// Low-level module: CreditCardProcessor implements
PaymentProcessor
class CreditCardProcessor implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing credit card payment of $" + amount);
    }
}

// Another low-level module: PaypalProcessor implements
PaymentProcessor
class PaypalProcessor implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing PayPal payment of $" + amount);
    }
}
```

```
// High-level module: PaymentService depends on the
PaymentProcessor abstraction
class PaymentService {
    private PaymentProcessor processor;

    public PaymentService(PaymentProcessor processor) {
        this.processor = processor;
    }

    public void makePayment(double amount) {
        processor.processPayment(amount);
    }
}
```

# PRINCIPLES OF COMPONENT DESIGN



- ✓ If SOLID principles tell us how to arrange the bricks into walls, then the component principles tell us how to arrange the rooms into buildings.
- ✓ Large software systems, like large buildings, are built out of smaller components.
- ✓ Components are smallest entities that can be **deployed** as part of a system.
- ✓ In Java, they are jar files. In Ruby, they are gem files. In .Net, they are DLLs.
- ✓ well-designed components always retain the ability to be **independently deployable** and, therefore, **independently developable**.



# PRINCIPLES OF COMPONENT DESIGN



They are divided into two categories:

- ✓ **Principles of Component Cohesion** which focus on the **granularity of components** and help the developer **partition classes into components**
- ✓ **Principles of Component Coupling** which focus on the stability and **relationships between the components.**



# COMPONENT COHESION



Which classes belong in which components?

Three principles of component cohesion:

- **REP:** The Reuse/Release Equivalence Principle
- **CCP:** The Common Closure Principle
- **CRP:** The Common Reuse Principle



# THE REUSE/RELEASE EQUIVALENCE PRINCIPLE

---



- ✓ Classes and modules that are grouped together into a component should be releasable together.
- ✓ is closely related to versioning and release management in software development.
- ✓ It suggests that software components should be **organized and released** based on their **reuse potential**, meaning that components that are likely to be **reused together** should be **packaged and versioned** together.
- ✓ Reuse/Release Equivalence Principle emphasizes that the organization of components and their versioning should align with their patterns of reuse.

**What is reuse?**

# THE REUSE/RELEASE EQUIVALENCE PRINCIPLE

---



Example:

If you are building a logging library, you would want to bundle all related classes (e.g., `Logger`, `LogFormatter`, `LogWriter`) into a single, versioned library package.

This allows developers to add it to their projects as one reusable module, ensuring that they can get updates or bug fixes by upgrading to a new version without dealing with individual classes.

By adhering to the REP, software components become more manageable, maintainable, and reliable, improving the overall quality of the system and making reusability easier to achieve.

# THE COMMON CLOSURE PRINCIPLE

---



Gather into components those classes that change for the same reasons and at the same times.

Separate into different components those classes that change at different times and for different reasons

Just as the SRP says that a class should not contain multiples reasons to change, so the Common Closure Principle (CCP) says that a **component should not have multiple reasons to change**.

THE CLASSES IN A PACKAGE SHOULD BE CLOSED TOGETHER AGAINST THE SAME KINDS OF CHANGES. A CHANGE THAT AFFECTS A PACKAGE AFFECTS ALL THE CLASSES IN THAT PACKAGE.

# THE COMMON CLOSURE PRINCIPLE



Example:

Imagine you're building an invoicing system. You might have classes like: `Invoice`, `InvoiceCalculator`, `InvoicePrinter`

- ✓ These classes all relate to the invoicing functionality.
- ✓ If there's a change in the invoicing requirements (such as changes in tax calculations or formatting), it's likely to impact all three classes.
- ✓ According to CCP, these classes should be grouped in the same module or package, such as `invoice`, since they are likely to change together.

The CCP is about grouping classes together that are likely to **change for the same reason**. This makes the code more organized, easier to maintain, and less likely to break from changes.

# THE COMMON REUSE PRINCIPLE

---



THE CLASSES IN A PACKAGE ARE REUSED TOGETHER. IF YOU REUSE ONE OF THE CLASSES IN A PACKAGE, YOU REUSE THEM ALL.

Don't force users of a component to depend on things they don't need.

CRP is another principle that helps us to decide which classes and modules should be placed into a component. It states that classes and modules that tend to be reused together belong in the same component.

CRP says that classes that are not tightly bound to each other should not be in the same component.

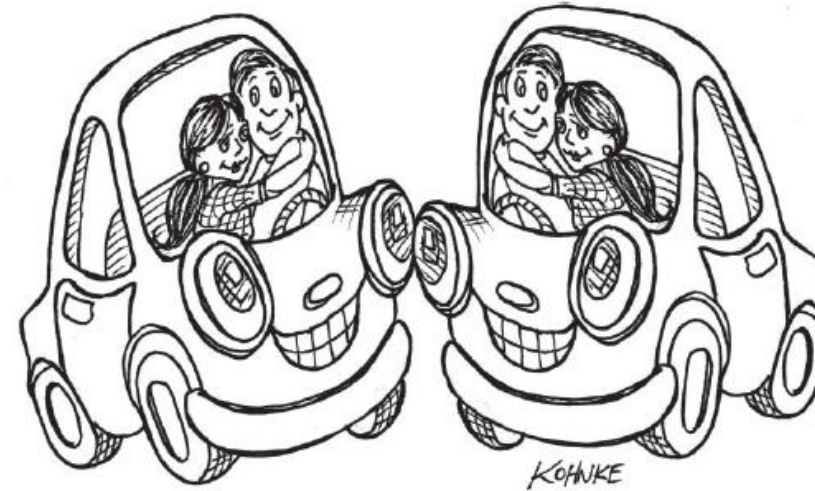
# COMPONENT COUPLING



## *Relationships between components*

Three principles of component Coupling:

- The Acyclic Dependencies Principle
- The Stable Dependencies Principle
- The Stable Abstractions Principle



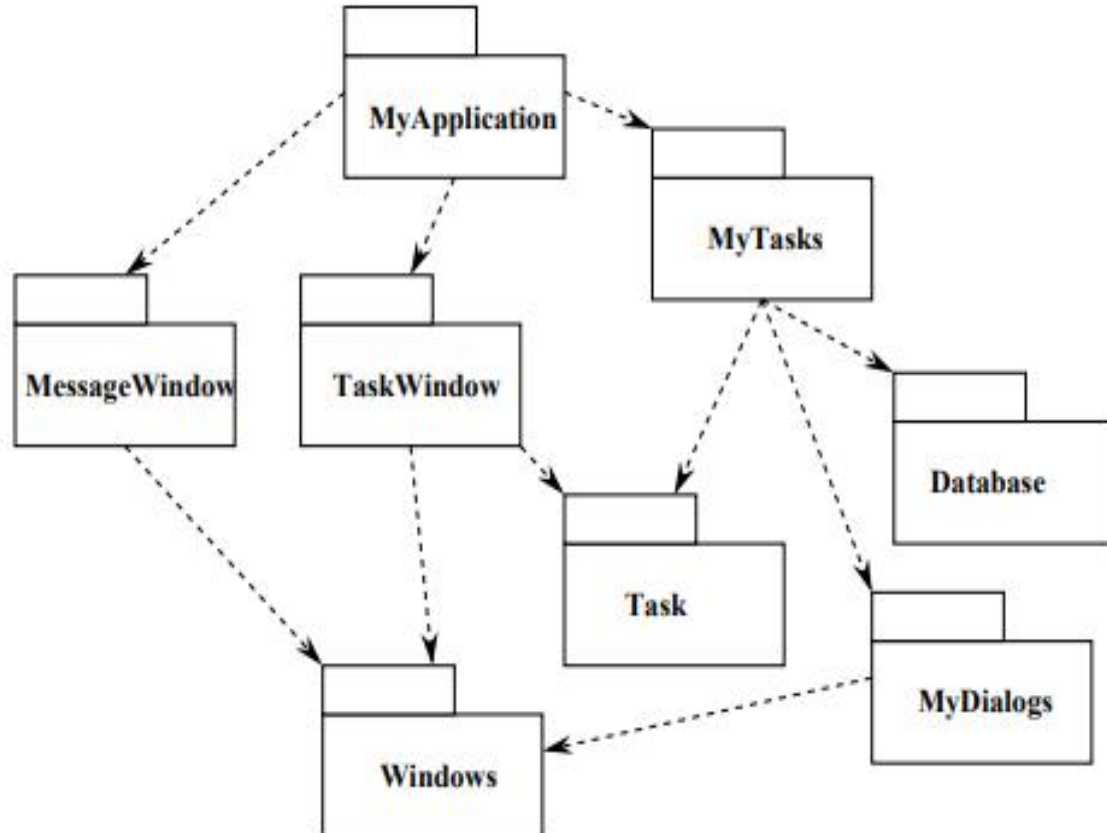
# THE ACYCLIC DEPENDENCIES PRINCIPLE



- ✓ THE DEPENDENCY STRUCTURE BETWEEN PACKAGES MUST BE A DIRECTED ACYCLIC GRAPH (DAG).
- ✓ THAT IS, THERE MUST BE NO CYCLES IN THE DEPENDENCY STRUCTURE.



## The Morning after Syndrome



Suppose **MyDialogs** need to make a new release.

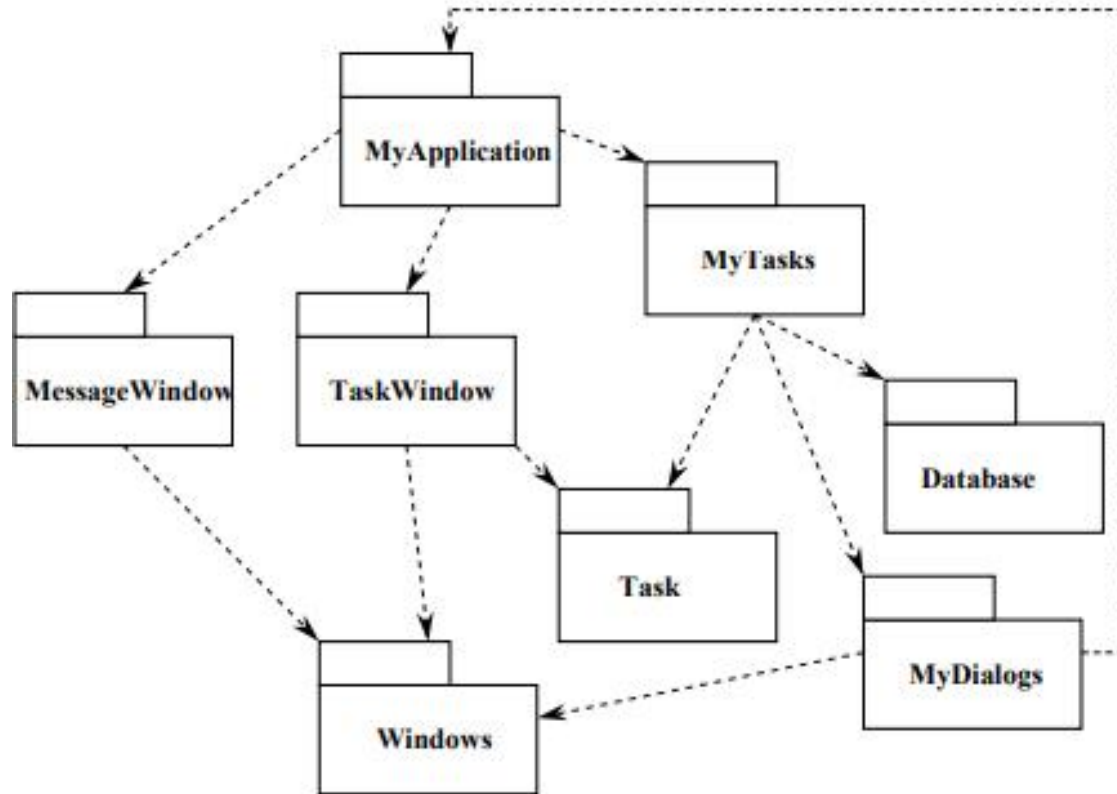
- who is affected by this release
  - MyTasks and MyApplication

The teams responsible for those packages will have to decide when they should integrate with the new release of MyDialogs.

other packages in the system don't know about MyDialogs and they don't care when it changes.

Means that the impact of releasing MyDialogs is relatively small.

# THE ACYCLIC DEPENDENCIES PRINCIPLE



What problems will occur with this new cycle

- To release MyTask, it must be compatible with Task, MyDialogs, Database, and Windows.
- But, with the cycle in place, it must now also be compatible with MyApplication, TaskWindow and MessageWindow.
- That is, MyTasks now depends upon every other package in the system.

MyDialogs suffers the same fate.

In fact, the cycle has the effect that MyApplication, MyTasks, and MyDialogs must always be released at the same time.

And all the engineers who are working in any of those packages will experience “the morning after syndrome” once again.

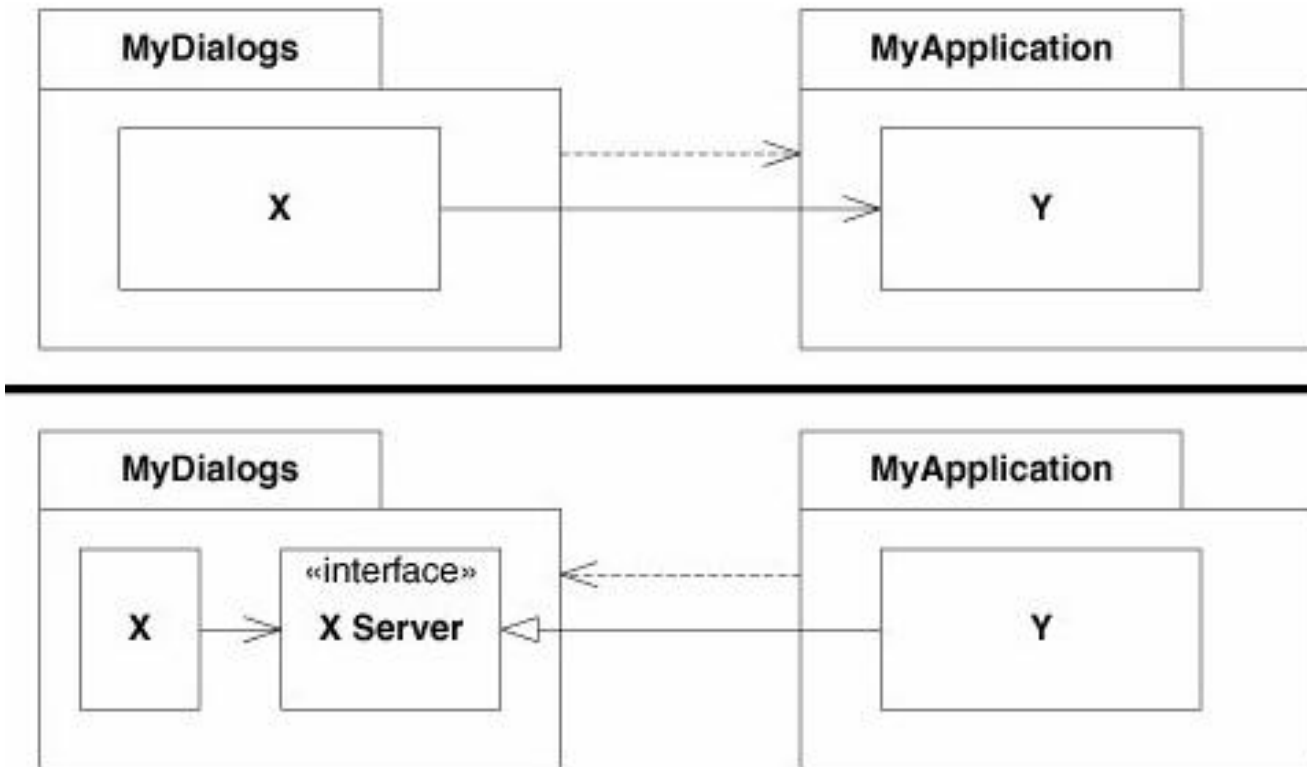
## How to Fix

1. Apply the Dependency Inversion Principle (DIP).
2. Create a new package that both MyDialogs and MyApplication depend upon. Move the class(es) that they both depend upon into that new package.

# THE ACYCLIC DEPENDENCIES PRINCIPLE



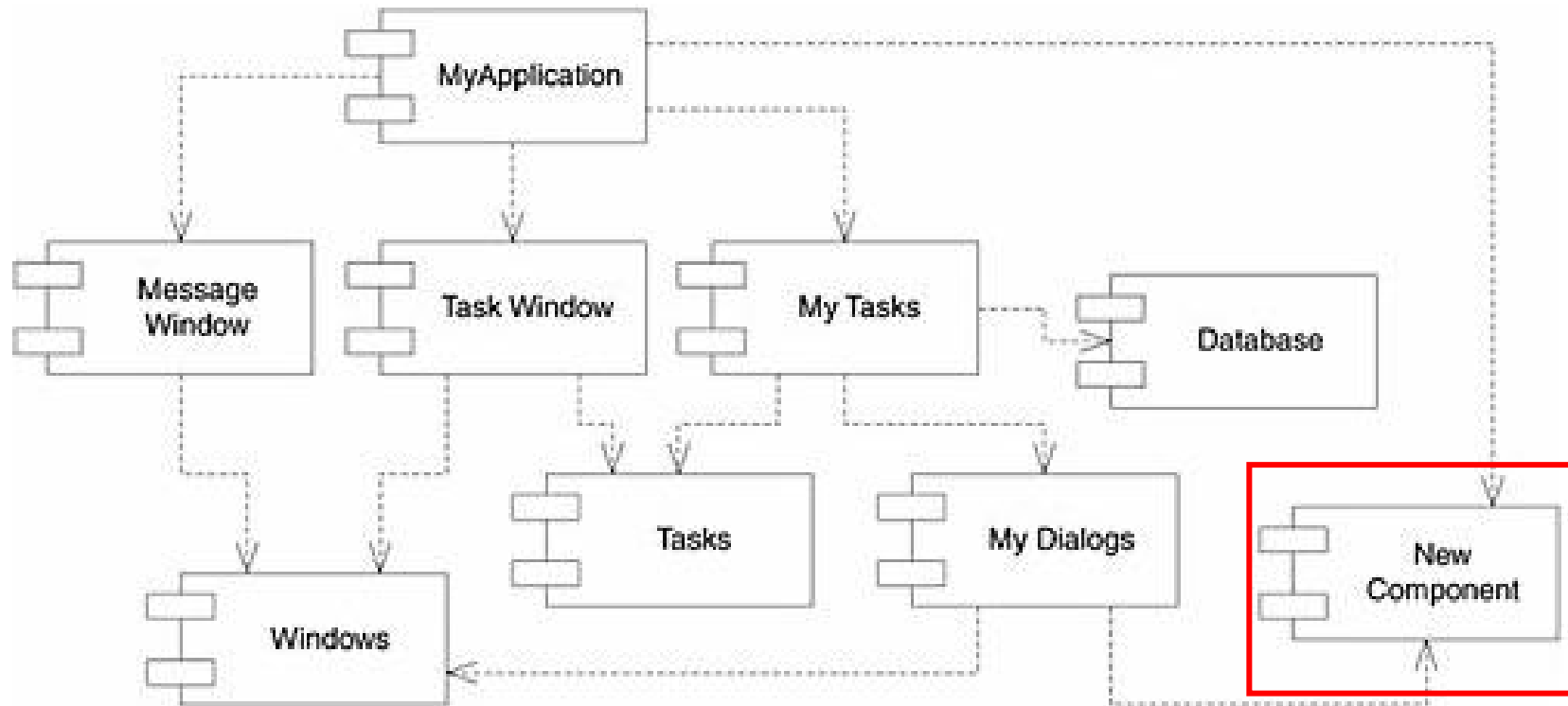
**Apply the Dependency-Inversion Principle (DIP)** we could create an **abstract base class** that has the **interface** that `MyDialogs` needs. We could then put that abstract base into `MyDialogs` and inherit it into `MyApplication`. This inverts the dependency between `MyDialogs` and `MyApplication`, thus breaking the cycle.



# THE ACYCLIC DEPENDENCIES PRINCIPLE



Create a new component that both `MyDialogs` and `MyApplication` depend on. Move the class(es) that they both depend on into that new component.



# THE STABLE DEPENDENCIES PRINCIPLE



*Depend in the direction of stability.*

*What is stability.*

Stability is related to the amount of work required to make a change. "Stable" roughly means "hard to change", whereas "instable" means "easy to change".

A component with lots of incoming dependencies is very stable because it requires a great deal of work to reconcile any changes with all the dependent components.

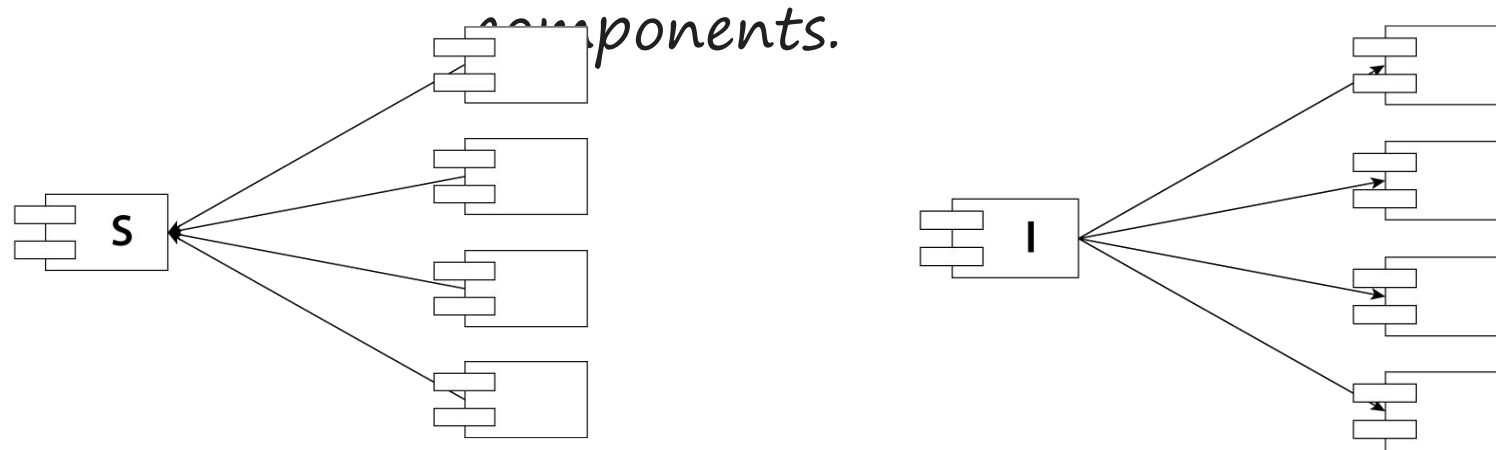


Figure shows S, which is a stable component. Four components depend on S, so it has four good reasons not to change. We say that S is responsible to those four components. Conversely, I depends on nothing, so it has no external influence to make it change. We say it is instable.

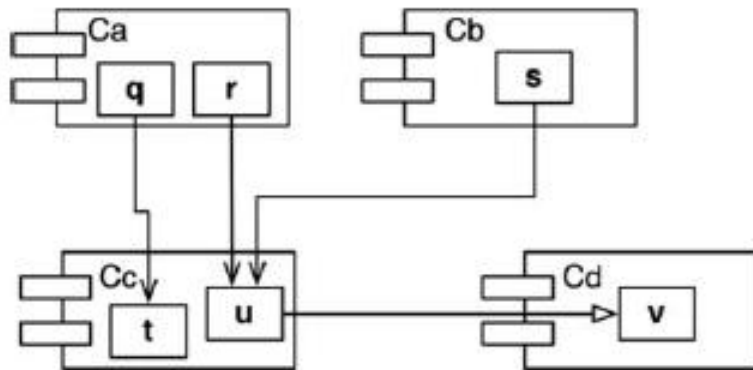
# CALCULATING COMPONENTS STABILITY



*Fan-in*: Incoming dependencies. This metric identifies the number of classes outside this component that depend on classes within the component.

- *Fan-out*: Outgoing dependencies. This metric identifies the number of classes inside this component that depend on classes outside the component.

- *I*: Instability:  $I = \text{Fan-out} / (\text{Fan-in} + \text{Fan-out})$ . This metric has the range [0, 1]. 0 indicates a maximally stable component. 1 indicates a maximally unstable component.



Let's say we want to calculate the stability of the component *Cc*. We find that there are three classes outside *Cc* that depend on classes in *Cc*. Thus, *Fan-in* = 3. Moreover, there is one class outside *Cc* that classes in *Cc* depend on. Thus, *Fan-out* = 1 and  $I = 1/4$ .

*Not All Components Should Be Stable*

# THE STABLE ABSTRACTIONS PRINCIPLE



The more stable a component is, the more abstract it should be

The Stable Abstractions Principle (SAP) sets up a relationship between stability and abstractness.

if a component is to be stable, it should consist of interfaces and abstract classes so that it can be extended.

How abstract a component is, is simply the ratio of interfaces and abstract classes in a component to the total number of classes in the component.

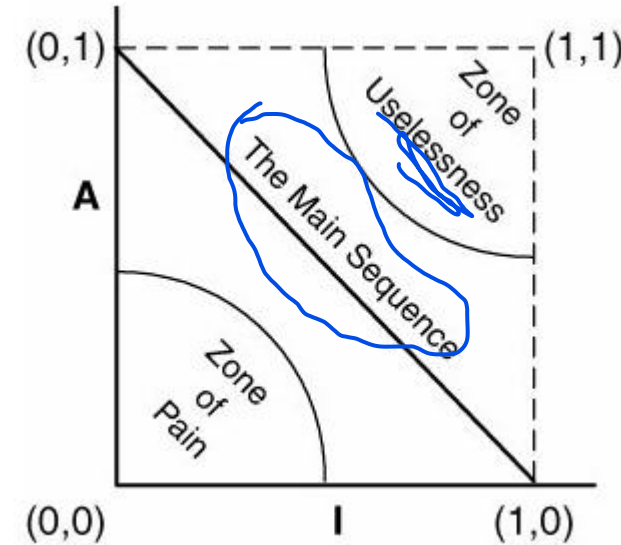
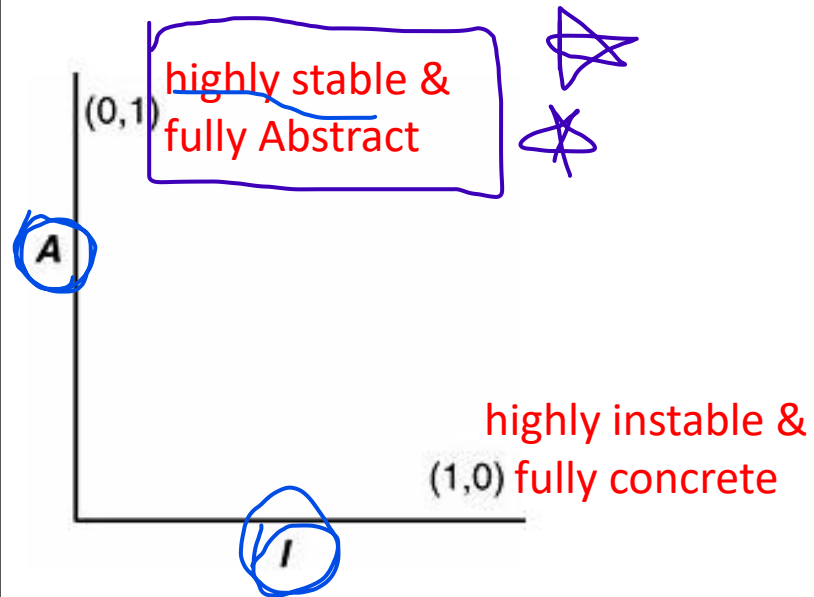
$N_c$ : The number of classes in the component.

$N_a$ : The number of abstract classes and interfaces in the component.

$A$ : Abstractness.  $A = N_a / N_c$ .

The  $A$  metric ranges from 0 to 1. A value of 0 implies that the component has no abstract classes at all. A value of 1 implies that the component contains nothing but abstract classes.

# RELATIONSHIP BETWEEN STABILITY (I) AND ABSTRACTNESS (A)



Consider a component in the area near (0,0). This component is highly stable and concrete. Such a component is not desirable, because it is rigid. It cannot be extended, because it is not abstract. And it is very difficult to change, because of its stability.

# SUMMARY



The first three package principles are about package *cohesion*

|     |                                         |                                                              |
|-----|-----------------------------------------|--------------------------------------------------------------|
| REP | The Release Reuse Equivalency Principle | <i>The granule of reuse is the granule of release.</i>       |
| CCP | The Common Closure Principle            | <i>Classes that change together are packaged together.</i>   |
| CRP | The Common Reuse Principle              | <i>Classes that are used together are packaged together.</i> |

Other three principles are about the couplings between packages

|     |                                    |                                                              |
|-----|------------------------------------|--------------------------------------------------------------|
| ADP | The Acyclic Dependencies Principle | <i>The dependency graph of packages must have no cycles.</i> |
| SDP | The Stable Dependencies Principle  | <i>Depend in the direction of stability.</i>                 |
| SAP | The Stable Abstractions Principle  | <i>Abstractness increases with stability.</i>                |